



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Hoja de Referencia: CSS Moderno (Nesting, Grid y Container Queries)

Esta guía rápida le proporciona los conceptos clave, la sintaxis estándar y ejemplos prácticos necesarios para resolver la maquetación y estilos responsivos del **Laboratorio 2**.

1. Custom Properties (Variables CSS)

Las variables CSS permiten centralizar y reutilizar tokens de diseño (colores, fuentes, espaciados) en la cascada:

```
/* Declaración en el ámbito global */
:root {
  --primary: #0d9488;
  --primary-hover: #0f766e;
  --radius: 8px;
  --transition: all 0.3s ease;
}
```

```

/* Uso de variables */
.btn {
  background-color: var(--primary);
  border-radius: var(--radius);
  transition: var(--transition);
}

/* Uso con valor por defecto (fallback) */
.card {
  background-color: var(--bg-card, #1e293b);
}

```

2. Nesting Nativo (Anidamiento de Selectores)

El anidamiento nativo de CSS reduce la duplicación de código agrupando los selectores relacionados dentro de sus contenedores:

```

/* Sintaxis con anidamiento nativo */
.formulario-registro {
  background-color: var(--bg-card);
  padding: 2rem;

  /* Anidar inputs */
  input, select {
    width: 100%;
    padding: 0.75rem;
    border: 1px solid var(--border-color);

    /* Selector del padre (&) para estados */
    &:focus {
      outline: none;
      border-color: var(--primary);
      box-shadow: 0 0 5px rgba(13, 148, 136, 0.4);
    }

    &::placeholder {
      color: var(--text-muted);
    }
  }
}

```

```
/* Anidar botones */
button {
  background-color: var(--primary);
  color: white;

  &:hover {
    background-color: var(--primary-hover);
  }
}
```

3. CSS Grid Layout Elástico (Sin Media Queries)

Para crear retículas elásticas y responsivas que se adapten a cualquier dispositivo sin usar puntos de ruptura rígidos, combine `auto-fit` y `minmax()`:

```
.catalogo-grid {
  display: grid;
  /* Columnas auto-ajustables con un ancho mínimo de 280px */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
  margin-top: 2rem;
}
```

4. Container Queries (@container)

A diferencia de las Media Queries (que leen el ancho total de la pantalla), las **Container Queries** le permiten cambiar el estilo de un componente según el espacio físico real de su contenedor inmediato:

Paso 1: Definir el contexto del contenedor

Para que un elemento sea consultado, debe marcarse como un contenedor estableciendo su tipo:

```

/* El elemento que aloja a <tarjeta-libro> actúa como contenedor */
.tarjeta-wrapper {
  container-type: inline-size;
  container-name: tarjeta-cont;
}

```

Paso 2: Aplicar la consulta de contenedor

Dentro de los estilos de la tarjeta (en su Shadow DOM o estilos locales), defina los cambios visuales según el ancho de ese contenedor:

```

/* Estilo base (por defecto: tarjeta grande/horizontal) */
.libro-card {
  display: flex;
  flex-direction: row;
  gap: 1.5rem;
  padding: 1.5rem;
}

/* Adaptación cuando el contenedor es estrecho (menos de 350px) */
@container tarjeta-cont (max-width: 350px) {
  .libro-card {
    flex-direction: column; /* Apilar verticalmente */
    padding: 1rem;

    .descripcion {
      display: none; /* Ocultar detalles secundarios */
    }
  }
}

```

■ Consejos de Implementación

- **Identifique el contenedor:** Para que una Container Query funcione, el elemento que consulta (`@container`) debe estar *dentro* de un ancestro que tenga declarado el `container-type`. No puede declarar `container-type` y `@container` sobre el mismo elemento.

- **Shadow DOM y Variables CSS:** Aunque el Shadow DOM bloquea los selectores normales de CSS, las variables CSS (`var()`) sí penetran en él. Use variables en `:root` para dar estilo al Shadow DOM de sus componentes de forma centralizada.